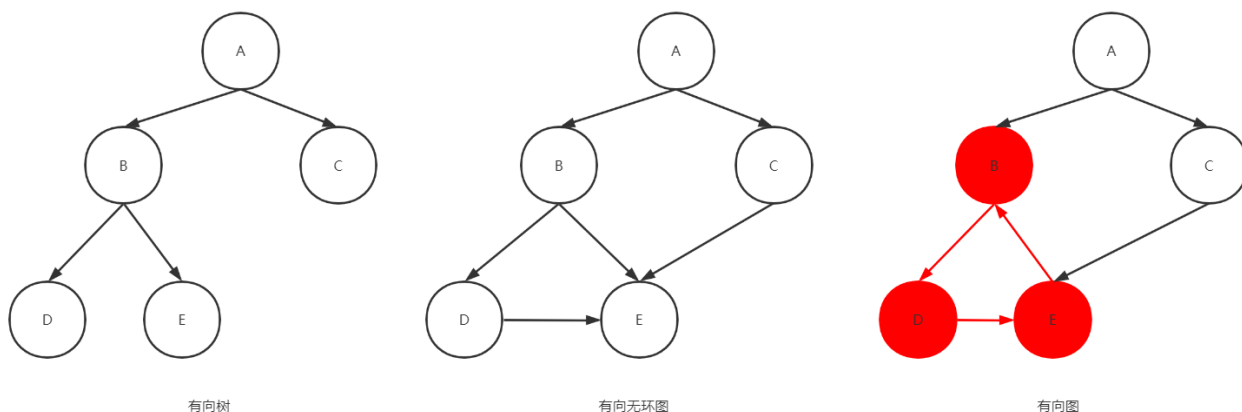


拓扑排序

基本概念

什么是有向无环图？

一个 **无环的有向图** 称为 **有向无环图**（Directed Acycline Graph），简称 DAG 图。（这不等于是说吗？）所以直接看图。



图中最左边的是有向树，中间的是有向无环图，最右则的是有向图（因为图中BED三个顶点之间构成一个有向环）。

什么是活动？

所有的工程或者某种流程都可以分为若干个小的工程或者阶段，我们称这些小的工程或阶段为“活动”。打个比方，如何把一只大象装到冰箱里，很简单，分三步。第一，打开冰箱门；第二，将大象装进去；第三，关上冰箱门。这三步中的每一步便是一个“活动”。

什么是AOV网？

在一个表示工程的有向图中，用 **顶点表示活动**，用**弧表示活动之间的优先关系的有向图** 称为**顶点表示活动的网**（Activity On Vertex Network），简称AOV网。

日常生活中，一项大的工程可以看作是由若干个子工程组成的集合，这些子工程之间必定存在一定的先后顺序，即某些子工程必须在其他的一些子工程完成后才能开始。

AOV网中的弧表示活动之间存在的某种制约关系，比如上面说到将大象装入冰箱，必须先打开冰箱门，才能将大象装进去，大象装进去才能关上冰箱门，从而完成我们的任务。还有一个经典的例子那就是选课，通常我们是学了C语言程序设计，才能学习数据结构，这里的制约关系就是课程之间的优先关系。（之后我们会讲一个完整的例子，大家就会更清楚AOV网）。

什么是拓扑序列?

设 $G=(V,E)$ 是一个具有 n 个顶点的有向图, V 中的顶点序列 $V_1,V_2,V_3,\dots,V_n$ 满足若从顶点 V_i 到 V_j 有一条路径, 则在顶点序列中顶点 V_i 必在顶点 V_j 之前。则我们称这样的顶点序列为一个拓扑序列。

什么是拓扑排序呢?

所以说, 所谓的拓扑排序, 其实就是对一个有向无环图构造拓扑序列的过程。当然这里的说法不够正式, 也是为了理解方便, 拓扑排序的官方定义是这样的:

由某个集合上的一个偏序得到该集合上的一个全序的操作过程称为拓扑排序。

那什么是偏序, 什么又是全序呢, 我希望各位同学上课的时候, 或者如果有时间。好好听《离散数学》的课, 你就可以严格地按照数学的方式对拓扑排序进行推导了。题外话, 《离散数学》真的很重要, 和所有计算机基础都是一样重要, 我一个同学的硕士论文就是用离散数学的知识和数据结构中的并查集解决的, 还望各位同学珍重奥。

算法思想

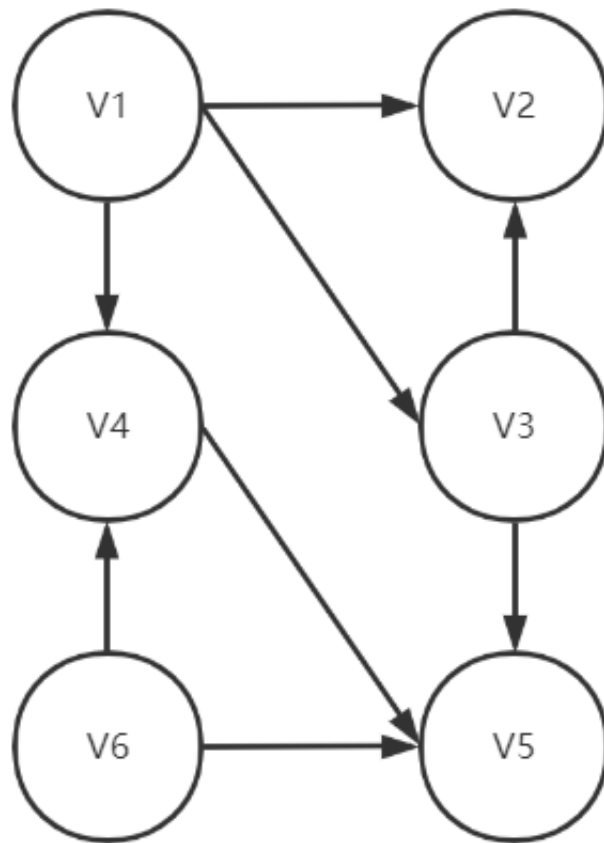
拓扑排序的算法步骤很简单, 就是两步:

- (1) 在有向图中选一个没有前驱的顶点且输出之。
- (2) 从图中删除该顶点和所有以它为尾的弧。

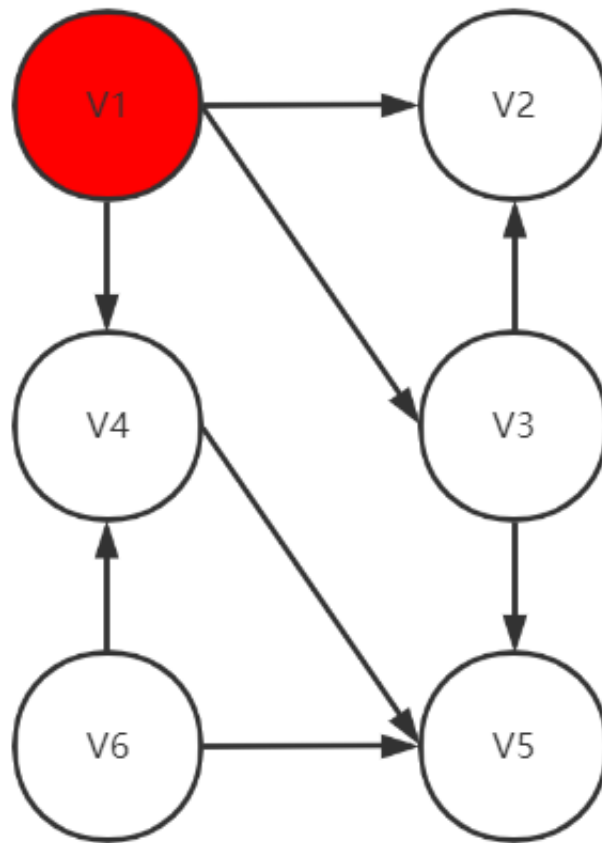
重复上述两步, 直至全部顶点均已输出, 或者当前图不存在无前驱的顶点为止, 后一种情况说明有向图中存在环。

为了清楚地理解拓扑排序思想, 我们分别采用有向无环图和有向有环图进行举例说明。

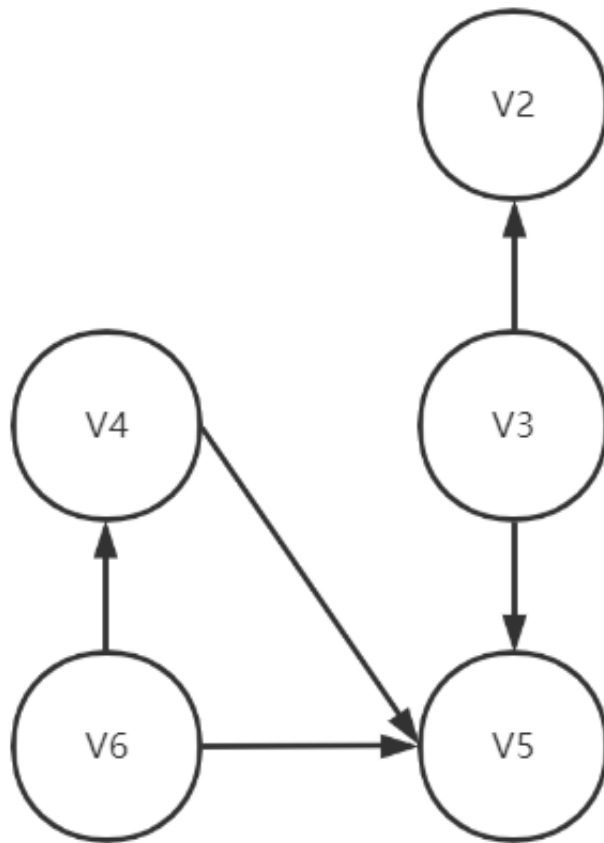
有向无环图



第一步：在有向图中选择一个没有前驱的顶点并输出；观察图中的顶点，发现顶点V1和顶点V6都是没有前驱的顶。假设我们先输出顶点V1（当然也可以先输出V6，从此处也就可以看出拓扑序列可以有多个）。

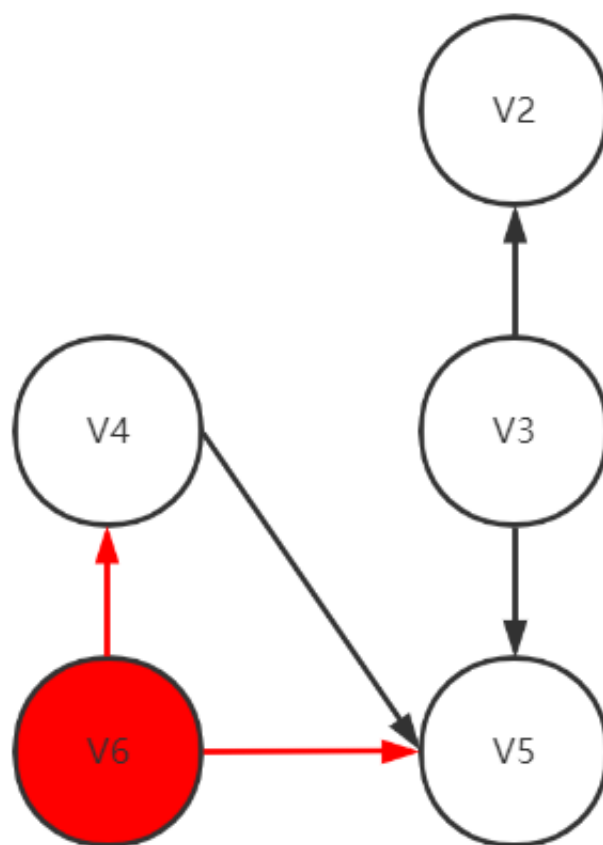


第二步：从图中删除顶点V1和所有以它为尾的弧。

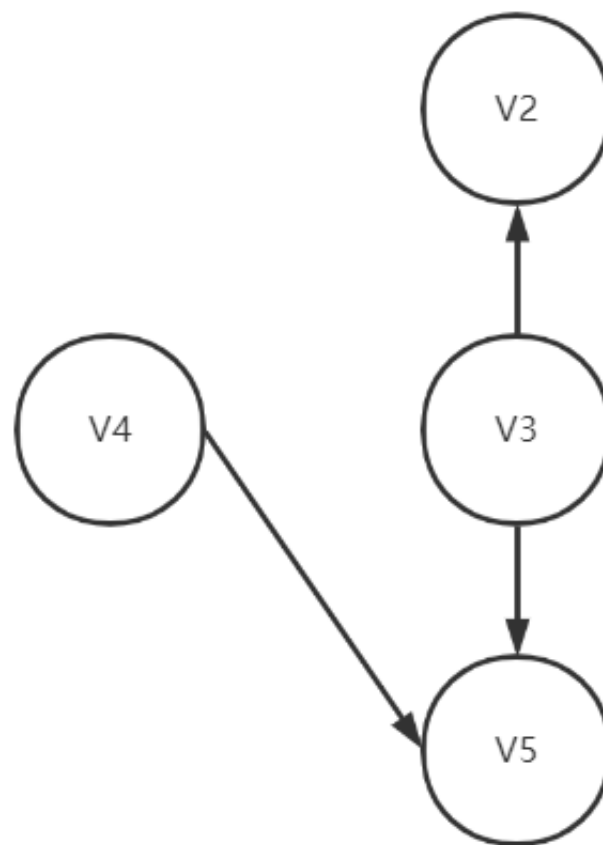


之后的步骤就是重复一二两步，我们接着看。

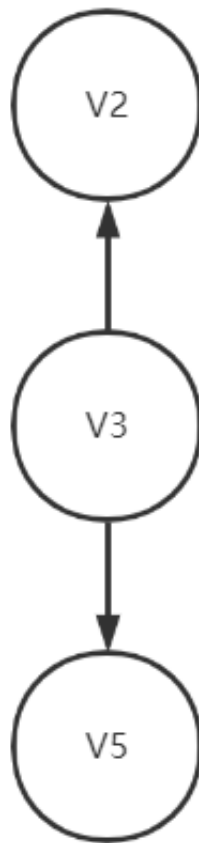
第三步：在有向图中选择一个没有前驱的顶点并输出；图中没有前驱的顶点为V6和顶点V3（同样的道理，我们可以选择这两个顶点的任何一个，假设我们选择顶点V6）。



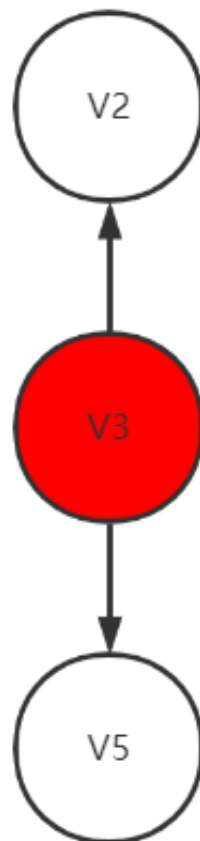
第四步：删除顶点V6和所有以它为尾的弧。

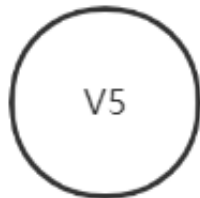
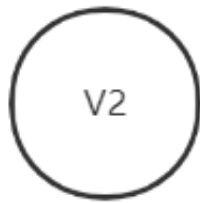


第五步：在有向图中选择一个没有前驱的顶点并输出；图中没有前驱的顶点为V4和顶点V3（同样的道理，我们可以选择这两个顶点的任何一个，假设我们选择了顶点V4）。删除顶点V4和所有以它为尾的弧。

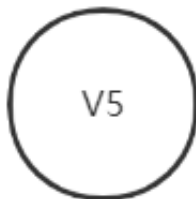


重复操作：在有向图中选择一个没有前驱的顶点并输出；图中没有前驱的顶点为V3。删除顶点V3和所有以它为尾的弧。





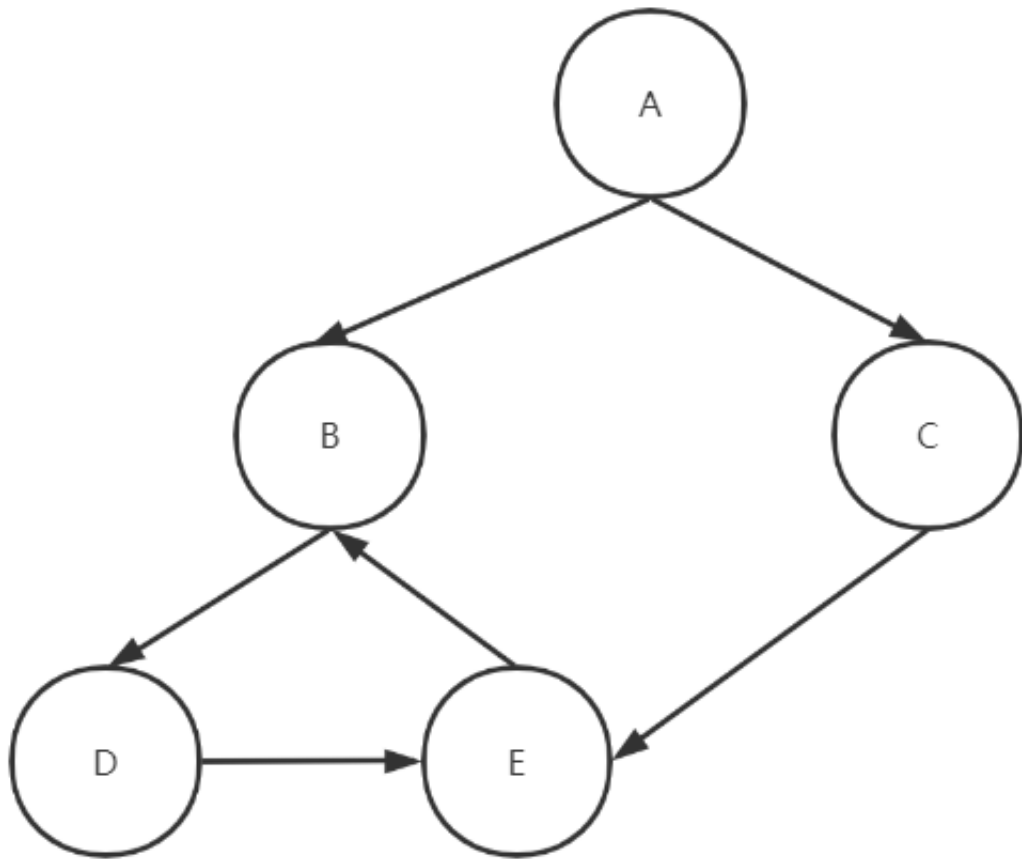
在有向图中选择一个没有前驱的顶点并输出；图中没有前驱的顶点为V2和 V5（同样的道理，我们可以选择这两个顶点的任何一个，假设我们选择了顶点V2）。删除顶点 V2和所有以它为尾的弧。



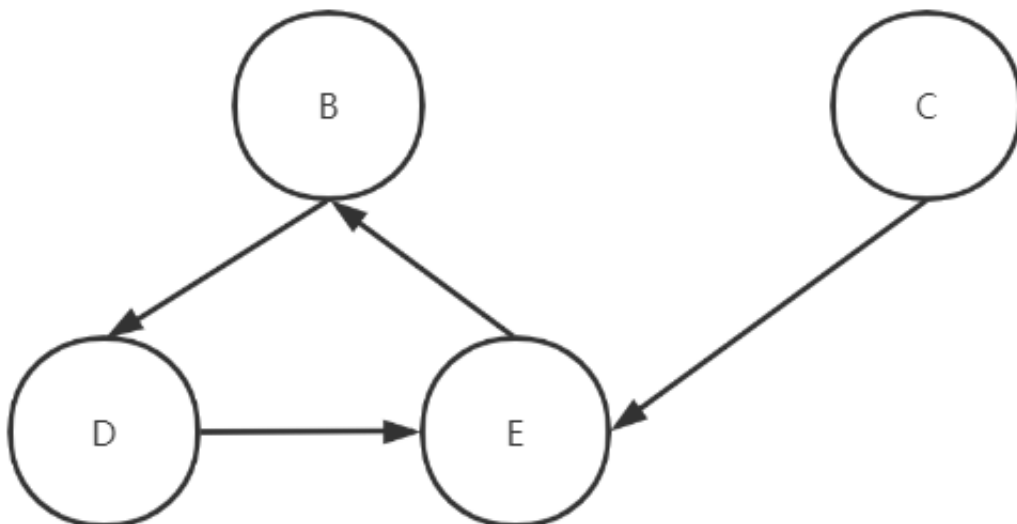
在有向图中选择一个没有前驱的顶点并输出；图中没有前驱的顶点为V5，选择并输出，此时所有的顶点均已经输出，算法结束，我们就得到了下图中的一个**拓扑序列**，整个过程便叫做**拓扑排序**。

有向有环图

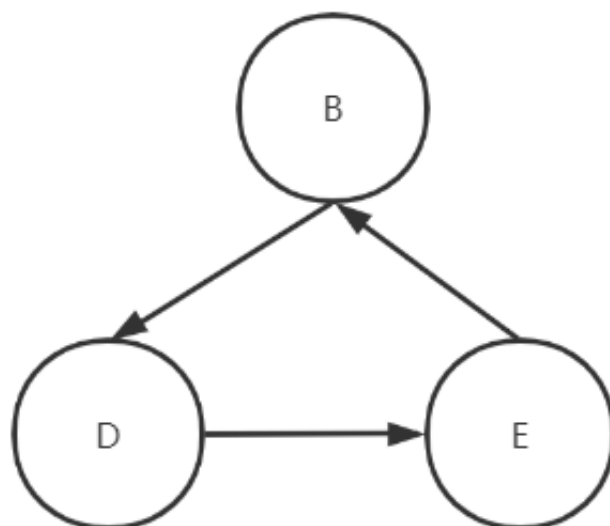
其实过程和之前类似，只是给大家提供一个思路，如果面试官问你，如何判断一个有向图中是否存在环时，你应该第一反应想到的就是拓扑排序，为什么拓扑排序可以判断一个有向图中是否存在环呢？我们看栗子。



第一步：在有向图中选择一个没有前驱的顶点并输出；图中没有前驱的顶点为A。删除顶点A和所有以它为尾的弧。



第二步：在有向图中选择一个没有前驱的顶点并输出；图中没有前驱的顶点为C。删除顶点C和所有以它为尾的弧。



第三步：在有向图中选择一个没有前驱的顶点并输出；发现当前图不存在无前驱的顶点，但拓扑序列中并未输出所有的顶点，所以剩下的顶点构成了环，也证明了该有向图存在环。

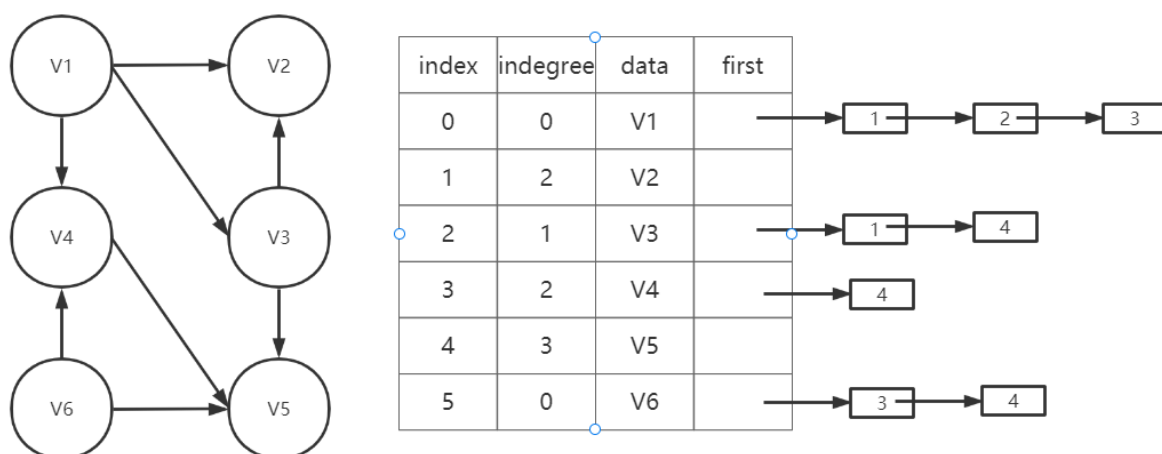
思想的实现

针对于拓扑排序思想篇提到的两步操作，我们采用邻接表作为有向图的存储结构，并且在头结点中增加一个存放顶点入度的数组（indegree）。入度为零的顶点即为没有前驱的顶点，删除顶点及以它为尾的弧的操作，则则可换以弧头顶点的入度减1来实现。

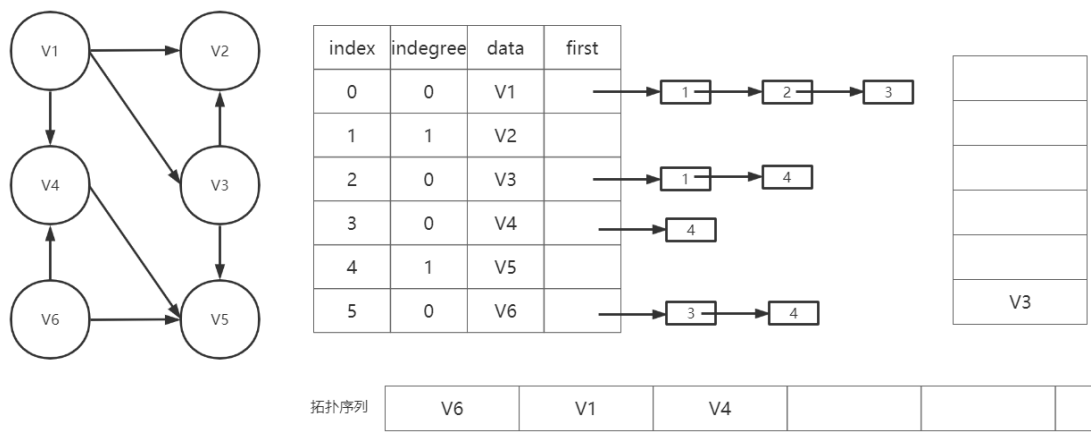
为了避免重复检查入度为零的顶点，可另设一个栈暂存所有入度为零的顶点。

为了清晰地呈现拓扑排序的实现，我们还是以上面提到的有向无环图为例进行讲解。

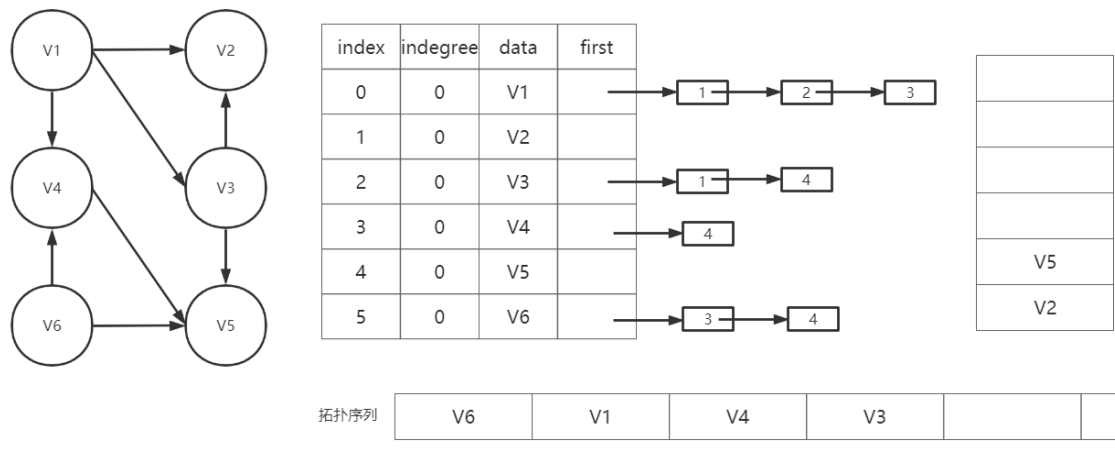
图的邻接表表示。



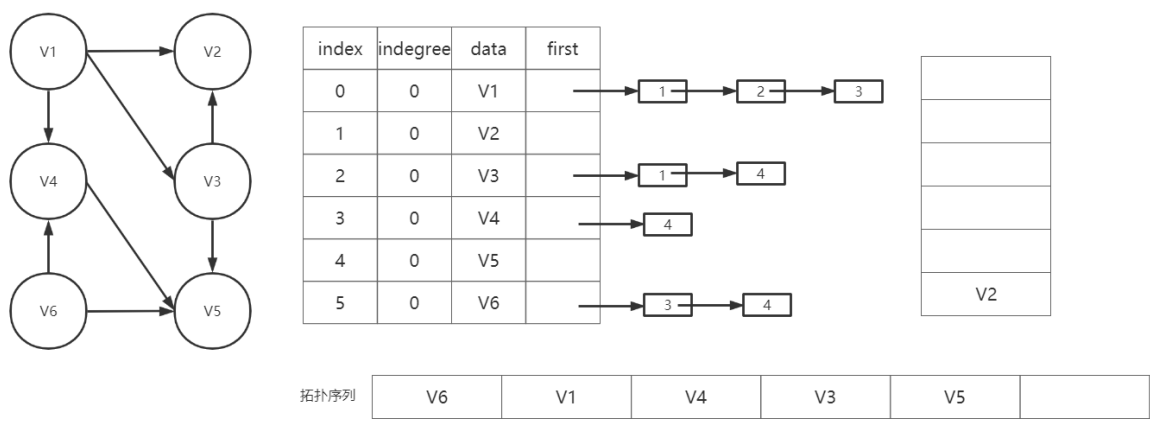
第一步：遍历所有顶点，将入度为0的顶点入栈，即分别将顶点V1和顶点V6入栈。



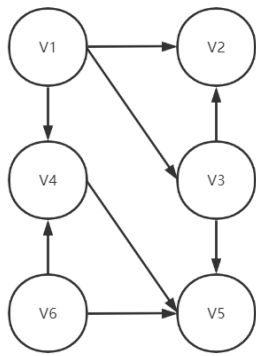
第五步：弹出栈顶顶点V3并输出，然后遍历顶点V3的邻接顶点，即 index == 1 和 index == 4 的顶点；将 index == 1 的顶点V2的入度减 1 等于 0，则入栈；将 index == 4 的顶点V5的入度减 1 等于 0，则入栈；



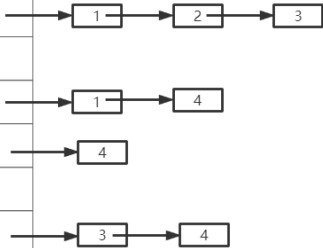
第六步：弹出栈顶顶点V5并输出，顶点V5没有后继顶点；



第七步：弹出栈顶顶点V2并输出，顶点V2没有后继顶点；



index	indegree	data	first
0	0	V1	
1	0	V2	
2	0	V3	
3	0	V4	
4	0	V5	
5	0	V6	



拓扑序列	V6	V1	V4	V3	V5	V2
------	----	----	----	----	----	----

总结

拓扑排序在实际的生产中很常见，最经典的例子就是课程表，当然还可以进行代码的依赖关系处理等等，此外，拓扑排序还是之后要讲的关键路径的基础，大家加油。